Unicode in the Library, Part 1: UTF Transcoding

Document #:	P2728R12 [Latest] [Status]
Date:	2026-05-12
Project:	Programming Language C++
Audience:	SG-16 Unicode SG-9 Ranges LEWG

Reply-to:	Eddie Nolan <eddiejnolan@gmail.com>

Contents

- 1 High-Level Overview
- 2 UTF Primer
- 3 Existing Standard UTF Interfaces in C and C++
  - 3.1 C
  - 3.2 C++
- 4 Replacing Ill-Formed Subsequences with ""
- 5 Design Overview
  - 5.1 Transcoding Views
    - 5.1.1 `utf_transcoding_error`
  - 5.2 Code Unit Views
- 6 Additional Examples
  - 6.1 Transcoding a UTF-8 String Literal to a `std::u32string`
  - 6.2 Sanitizing Potentially Invalid Unicode
  - 6.3 Returning the Final Non-ASCII Code Point in a String, Transcoding Backwards Lazily
  - 6.4 Transcoding Strings and Throwing a Descriptive Exception on Invalid UTF
  - 6.5 Changing the Suits of Unicode Playing Card Characters
  - 6.6 Handling Byte Offsets and Endianness
- 7 Dependencies
- 8 Implementation Experience
- 9 Wording
  - 9.1 Additional Helper Concepts
  - 9.2 Transcoding views
    - 24.7.? Transcoding views [range.transcoding]

- 9.3 Code unit adaptors
  - 24.7.? Code unit adaptors [range.codeunitadaptor]
- 9.4 Feature test macro

- 10 Design Discussion and Alternatives
  - 10.1 CPO Rejection of String Literals
  - 10.2 The `_or_error` Views Are Basis Operations for Other Error Handling Behaviors
  - 10.3 Why We Don't Cache `begin()`
  - 10.4 Optimizing for Double-Transcoding

- 11 Changelog
  - 11.1 Changes since R11
  - 11.2 Changes since R10
  - 11.3 Changes since R9
  - 11.4 Changes since R8
  - 11.5 Changes since R7
  - 11.6 Changes since R6
  - 11.7 Changes since R5
  - 11.8 Changes since R4
  - 11.9 Changes since R3
  - 11.10 Changes since R2
  - 11.11 Changes since R1
  - 11.12 Changes since R0

- 12 Relevant Polls/Minutes
  - 12.1 SG9 review of P2728R9 on 2025-11-06 during Kona 2025
  - 12.2 Unofficial SG9 review of P2728R7 during Wrocław 2024
  - 12.3 SG16 review of P2728R6 on 2023-09-13 (Telecon)
  - 12.4 SG16 review of P2728R6 on 2023-08-23 (Telecon)
  - 12.5 SG9 review of D2728R4 on 2023-06-12 during Varna 2023
  - 12.6 SG16 review of P2728R0 on 2023-04-12 (Telecon)
  - 12.7 SG16 review of P2728R0 on 2023-03-22 (Telecon)

- 13 Special Thanks

- 14 References

§ 1 High-Level Overview

This paper introduces views and ranges for transcoding between UTF formats:

```
static_assert((u8"🙂" | views::to_utf32 | ranges::to<u32string>()) == U"🙂");
```

It handles errors by replacing invalid subsequences with  :

```
static_assert((u8"🙂" | views::take(3) | to_utf32 | ranges::to<std::u32string>()) ==
U"🙂");
```

And by providing `or_error` views that provide `std::expected`:

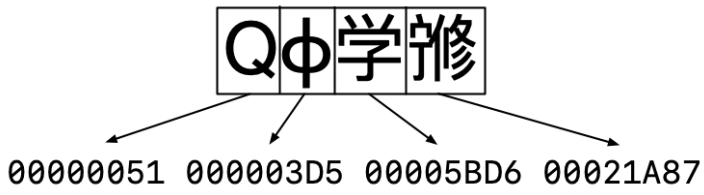
```
static_assert(
  *(u8"🙂" | views::take(3) | views::to_utf32_or_error).begin() ==
  unexpected{utf_transcoding_error::truncated_utf8_sequence});
```

§ 2 UTF Primer

If you're already familiar with Unicode, you can skip this section.

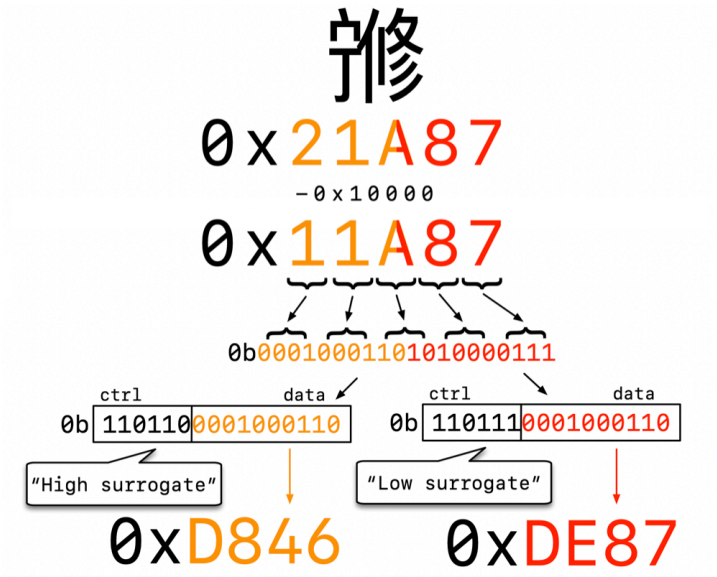
The Unicode standard maps *abstract characters* to *code points* in the *Unicode codespace* from `0` to `0x10FFFF`. Unicode text forms a *coded character sequence*, "an ordered sequence of one or more code points." [Definitions]

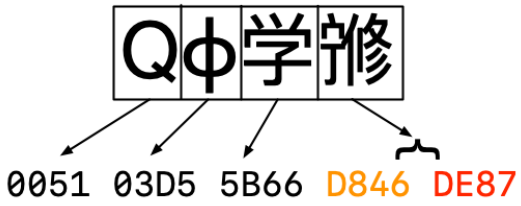
The simplest way of encoding code points is UTF-32, which encodes code points as a sequence of 32-bit unsigned integers. The building blocks of an encoding are *code units*, and UTF-32 has the most direct mapping between code points and code units.



Any values greater than `0x10FFFF` are rejected by validators for being outside the range of valid Unicode.

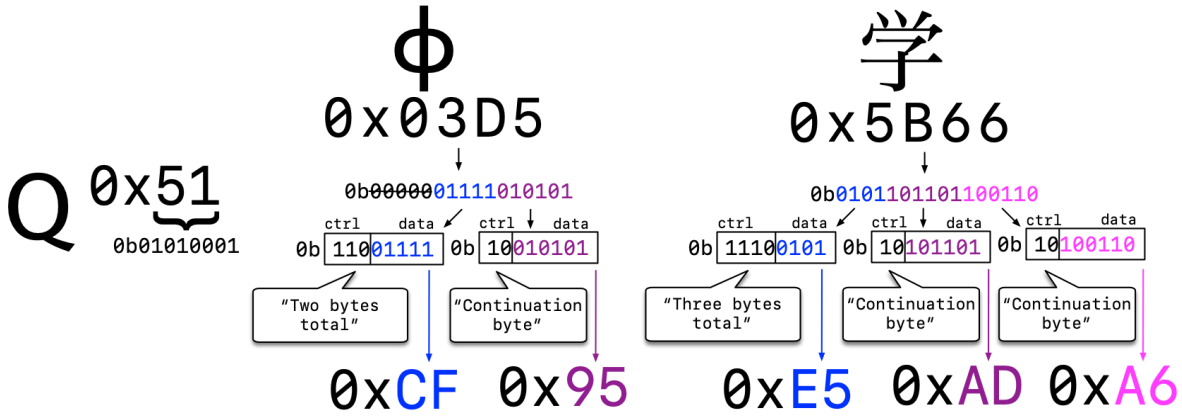
Next is UTF-16, which exists for the historical reason that the Unicode codespace used to top out at `0xFFFF`. Code points outside this range are represented using *surrogates*, a reserved area in codespace which allows combining the low 10 bits of two code units to form a single code point.

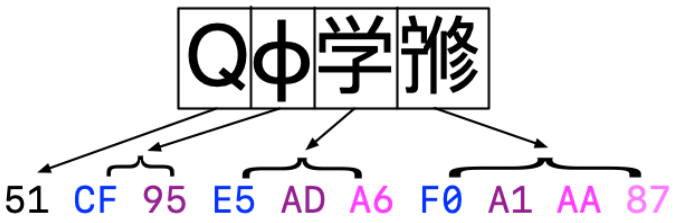




UTF-16 is rendered invalid by improper use of surrogates: a high surrogate not followed by a low surrogate or a low surrogate not preceded by a high surrogate. Note that the presence of any surrogate code points in UTF-32 is also invalid.

Finally, UTF-8, the most ubiquitous and most complex encoding. This uses 8-bit code units. If the high bit of the code unit is unset, the code unit represents its ASCII equivalent for backwards compatibility. Otherwise the code unit is either a start byte, which describes how long the subsequence is (two to four bytes long), or a continuation byte, which fills out the subsequence with the remaining data.





UTF-8 code unit sequences can be invalid for many reasons, such as a start byte not followed by the correct number of continuation bytes, or a UTF-8 subsequence that encodes a surrogate.

Transcoding in this context refers to the conversion of characters between these three encodings.

§ 3 Existing Standard UTF Interfaces in C and C++

§ 3.1 C

C contains an alphabet soup of transcoding functions in `<stdlib.h>`, `<wchar.h>`, and `<uchar.h>`. [Null-terminated multibyte strings]

This paper doesn't fully litigate these functions' flaws (see WG14 [N2902] for a more detailed explanation). Some of the issues users encounter include reliance on an internal global conversion state, reliance on the current setting of the global C locale, optimization barriers in one-code-unit-at-a-time function calls, and inadequate error handling that does not support replacement of invalid subsequences with `U+FFFD` as specified by Unicode.

Example:

```
setlocale(LC_ALL, "en_US.utf8");
char c[5] = {0};
const char16_t* w = u"\xd83d\xdd74";
mbstate_t state;
memset(&state, 0, sizeof(state));
c16rtomb(c, w[0], &state);
c16rtomb(c, w[1], &state);
const char* e = "\xf0\x9f\x95\xb4";
assert(strcmp(c, e) == 0);
```

§ 3.2 C++

C++'s existing transcoding functionality, other than the aforementioned functions it inherits from C, consists of the set of `std::codecvt` facets provided in `<locale>` and `<codecvt>`.

Example:

```
std::wstring_convert<std::codecvt_utf8<char32_t>, char32_t> conv;
std::string c = conv.to_bytes(U"🕴");
assert(c == "\xf0\x9f\x99\x82");
```

All of the Unicode-specific functionality in this header was deprecated in C++17, and [P2871R3] and [P2873R2] finally remove most of it in C++26. There are many concerns about these interfaces, particularly with respect to safety.

These functions throw exceptions on encountering invalid UTF. Unicode functions that use exceptions for error handling are a well-known footgun because users consistently invoke them on untrusted user input without handling the exceptions properly, leading to denial-of-service vulnerabilities.

An example of this anti-pattern (although not involving these specific functions) can be found in [CVE-2007-3917], where a multiplayer RPG server could be crashed by malicious users sending invalid UTF. Below is the patch: [wesnoth]

```
- msg = font::word_wrap_text(msg,font::SIZE_SMALL,map_outside_area().w*3/4);
+ try {
+     // We've had a joker who send an invalid utf-8 message to crash clients
+     // so now catch the exception and ignore the message.
+     msg = font::word_wrap_text(msg,font::SIZE_SMALL,map_outside_area().w*3/4);
+ } catch (utils::invalid_utf8_exception&) {
+     LOG_STREAM(err, engine) << "Invalid utf-8 found, chat message is ignored.\n";
+     return;
+ }
```

Because it doesn't use exceptions, the functionality proposed by this paper can serve as a safe, modern replacement for the deprecated and removed `codecvt` facets.

§ 4 Replacing Ill-Formed Subsequences with "U+FFFD"

When a transcoder encounters an invalid subsequence, the modern best practice is to replace it in the output with one or more `U+FFFD REPLACEMENT CHARACTER` characters (`U+FFFD`, `REPLACEMENT CHARACTER`). The methodology for doing so is described in §3.9.6 of the Unicode Standard v17.0, Substitution of Maximal Subparts [Substitution].

For UTF-32 and UTF-16, each invalid code unit is replaced by an individual `U+FFFD REPLACEMENT CHARACTER` character.

For UTF-8, the same rule applies except if "a sequence of two or three bytes is a truncated version of a sequence which is otherwise well-formed to that point." In the latter case, the full two-to-three byte subsequence is replaced by a single `U+FFFD REPLACEMENT CHARACTER` character.

For example, UTF-8 encodes 🙂 as `0xF0 0x9F 0x99 0x82`.

If that sequence of bytes is truncated to just `0xF0 0x9F 0x99`, it becomes a single `U+FFFD REPLACEMENT CHARACTER` replacement character.

On the other hand, if the first byte of the four-byte sequence is changed from `0xF0` to `0xFF`, then it's replaced by four replacement characters, `U+FFFD U+FFFD U+FFFD U+FFFD`, because no valid UTF-8 subsequence begins with `0xFF`.

More subtly, the subsequence `0xED 0xA0` must be replaced with two replacement characters, `U+FFFD U+FFFD`, because any continuation of that subsequence can only result in a surrogate code point, so it can't prefix any valid subsequence.

Each of the proposed `to_utfN_view` views adheres to this specification. The `to_utfN_as_error` views also use this scheme but produce `unexpected<utf_transcoding_error>` values instead of replacement characters.

§ 5 Design Overview

§ 5.1 Transcoding Views

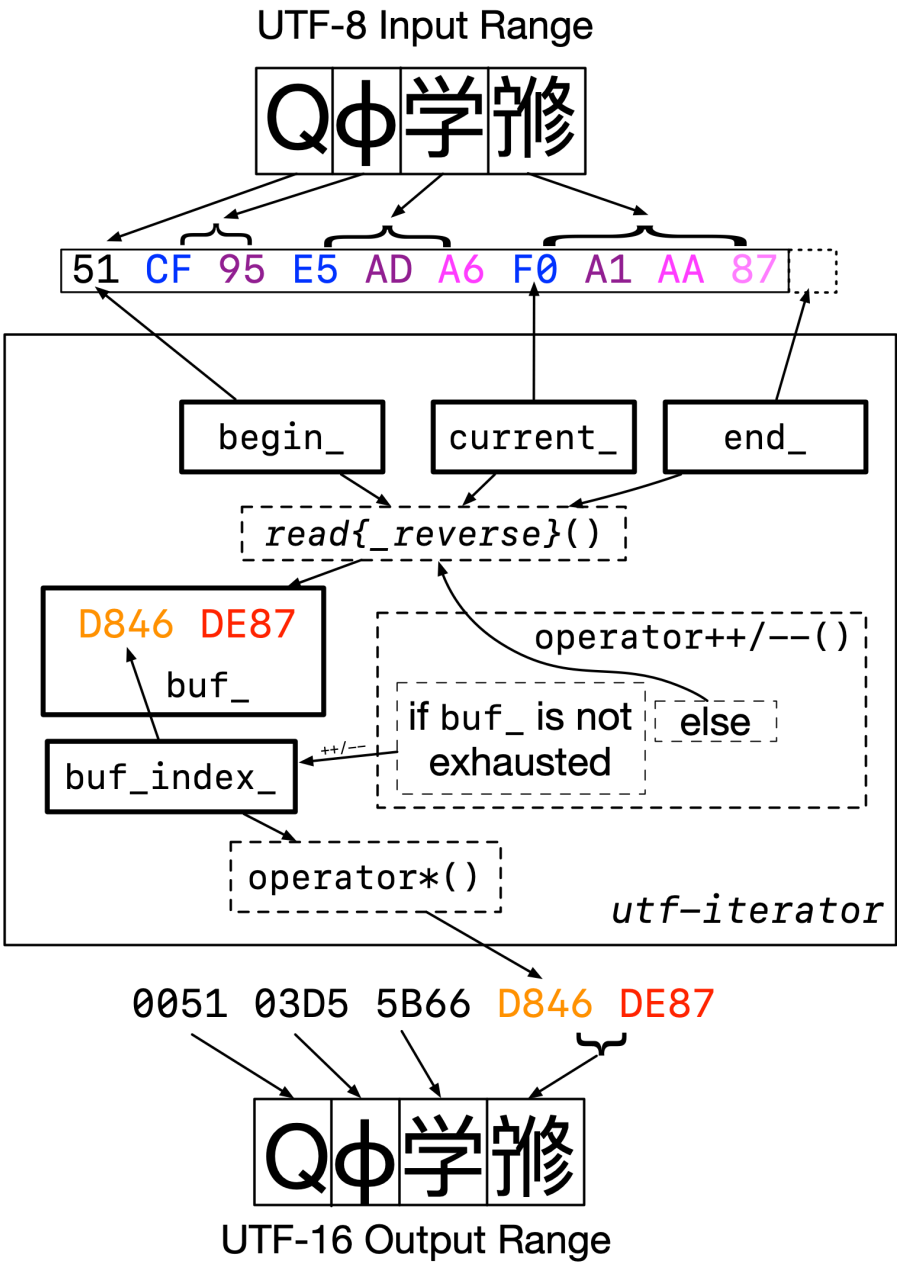
Invoking `begin()` or `end()` on a transcoding view constructs an instance of an exposition-only `to_utf_view::iterator` type.

The `to_utf_view::iterator` stores an iterator pointing to the start of the character it's transcoding, and a back-pointer to the underlying range in order to bounds check its beginning and end (which is required for correctness, not just safety).

The `to_utf_view::iterator` maintains a small buffer (`buf_`) containing between one and four code units, which comprise the current character in the target encoding.

It also maintains an index (`buf_index_`) into this buffer, which it increments or decrements when `operator++` or `operator--` is invoked, respectively. If it runs out of code units in the buffer, it reads more elements from the underlying view. `operator*` provides the current element of the buffer.

Below is an approximate block diagram of the iterator. Bold lines denote actual data members of the iterator; dashed lines are just function calls.



The `to_utf_view::iterator` is converting the string `Qϕ学𩸽` from UTF-8 to UTF-16. The user has iterated the view to the first UTF-16 code unit of the fourth character. `current_` points to the start of the fourth character in the input. `buf_` contains both UTF-16 code units of the fourth character; `buf_index_` keeps track of the fact that we're currently pointing to the first one. If we invoke `operator++` on the `to_utf_view::iterator`, it will increment `buf_index_` to point to the second code unit. On the other hand, if we invoke `operator--`, it will notice that `buf_index_` is already at the beginning and move backward from the fourth character to the third character by invoking `read_reverse()`. The `read()` and `read_reverse()` functions contain most of the actual transcoding logic, updating `current_` and filling `buf_` up with the transcoded characters.

Iterating a bidirectional transcoding view backwards produces, in reverse order, the exact same sequence of characters or `expected` values as are produced by iterating the view forwards.

§ 5.1.1 `utf_transcoding_error`

Each transcoding view, like `to_utf8_view`, which produces a range of `char8_t` and handles errors by substituting `�` replacement characters, has a corresponding `_or_error` equivalent, like `to_ut-`

`f8_view_or_error`, which produces a range of `expected<char8_t, utf_transcoding_error>` and handles errors by substituting `unexpected<utf_transcoding_error>`s.

`utf_transcoding_error` is an enumeration whose enumerators are:

- `truncated_utf8_sequence`
  - An ill-formed subsequence that matches the beginning of some well-formed sequence.
  - Example invalid code unit sequence: UTF-8 `0xE1 0x80`.
- `unpaired_high_surrogate`
  - Example invalid code unit sequence: UTF-16 `0xD800`.
- `unpaired_low_surrogate`
  - Example invalid code unit sequence: UTF-16 `0xDC00`.
- `unexpected_utf8_continuation_byte`
  - Example invalid code unit sequence: UTF-8 `0x80`.
- `overlong`
  - An overlong UTF-8 encoding.
  - Example invalid code unit sequence: UTF-8 `0xE0 0x80`.
- `encoded_surrogate`
  - Applies to both UTF-8 and UTF-32.
  - Example invalid code unit sequence: UTF-8 `0xED 0xA0`, UTF-32 `0x0000D800`.
- `out_of_range`
  - Applies to both UTF-8 and UTF-32
  - In UTF-8, this applies to `0xF4` if it is followed by a continuation byte greater than `0x8F`
  - In UTF-32, this is any code unit greater than `0x10FFFF`
  - Example invalid code unit sequence: UTF-8 `0xF4 0x90`, UTF-32 `0x110000`.
- `invalid_utf8_leading_byte`
  - In UTF-8, this applies to `0xC0`-`0xC1` and `0xF5`-`0xFF`.
  - Example invalid code unit sequence: UTF-8 `0xC0`.

An alternative approach to minimize the number of enumerators could merge `truncated_utf8_sequence` with `unpaired_high_surrogate` and merge `unexpected_utf8_continuation_byte` with `unpaired_low_surrogate`, but based on feedback, splitting these up seems to be preferred.

The table below compares the error handling behavior of the `to_utf16` and `to_utf16_or_error` views on various sample UTF-8 inputs from the "Substitution of Maximal Subparts" section of the Unicode standard: [SubstitutionExamples]

Table 3-8. U+FFFD for Non-Shortest Form Sequences

UTF-8 input	C0	AF	E0	80	BF	F0	81	82	41
to_utf16	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	0041
to_utf16_or_error	invalid_utf8_leading_byte	unexpected_utf8_continuation_byte	overlong	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	overlong	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	0041

Table 3-9. U+FFFD for Ill-Formed Sequences for Surrogates

UTF-8 input	E0	A0	80	ED	BF	BF	ED	AF	41
to_utf16	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	FFFD	0041
to_utf16_or_error	encoded_surrogate	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	encoded_surrogate	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	encoded_surrogate	unexpected_utf8_continuation_byte	0041

Table 3-10. U+FFFD for Other Ill-Formed Sequences

UTF-8 input	F4	91	92	93	FF	41	80	BF	42
to_utf16	FFFD	FFFD	FFFD	FFFD	FFFD	0041	FFFD	FFFD	0042
to_utf16_or_error	out_of_range	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	invalid_utf8_leading_byte	0041	unexpected_utf8_continuation_byte	unexpected_utf8_continuation_byte	0042

Table 3-11. U+FFFD for Truncated Sequences

UTF-8 input	E1	80	E2	F0	91	92	F1	BF	41
to_utf16	FFFD		FFFD	FFFD			FFFD		0041
to_utf16_or_error	truncated_utf8_sequence		truncated_utf8_sequence	truncated_utf8_sequence			truncated_utf8_sequence		0041

§ 5.2 Code Unit Views

SG16 has a goal to ensure that C++ standard library functions that expect UTF-encoded input do not accept parameters of type `char` or `wchar_t`, whose encodings are implementation-defined, and instead use `char8_t`, `char16_t`, and `char32_t`. These views follow that pattern.

Because virtually all UTF-8 text processed by C++ is stored in `char` (and similarly for UTF-16 and `wchar_t`), this means that we need a terse way to smooth over the transition for users. To do so, this paper introduces views for casting to the `charN_t` types: `as_char8_t`, `as_char16_t`, and `as_char32_t`.

These are syntactic sugar for producing a `std::ranges::transform_view` with an exposition-only transformation functor that performs the needed cast.

§ 6 Additional Examples

§ 6.1 Transcoding a UTF-8 String Literal to a `std::u32string`

```
std::u32string hello_world =
  u8"こんにちは世界"sv | std::views::to_utf32 | std::ranges::to<std::u32string>();
```

§ 6.2 Sanitizing Potentially Invalid Unicode

Note that transcoding to and from the same encoding is not a no-op; it must maintain the invariant that the output of a transcoding view is always valid UTF.

```
template <typename CharT>
std::basic_string<CharT> sanitize(CharT const* str) {
```

```
  return std::null_term(str) | std::views::to_utf<CharT> | std::ranges::to<std::ba-
      sic_string<CharT>>();
}
```

§ 6.3 Returning the Final Non-ASCII Code Point in a String, Transcoding Backwards Lazily

```
std::optional<char32_t> last_nonascii(std::ranges::view auto str) {
  for (auto c : str | std::views::to_utf32 | std::views::reverse
                    | std::views::filter([](char32_t c) { return c > 0x7f; })) {
    return c;
  }
  return std::nullopt;
}
```

§ 6.4 Transcoding Strings and Throwing a Descriptive Exception on Invalid UTF

(This assumes a reflection-based `enum_to_string` function.)

```
template <typename FromChar, typename ToChar>
std::basic_string<ToChar>  transcode_or_throw(std::basic_string_view<FromChar>  input)
      {
  std::basic_string<ToChar> result;
  auto view = input | std::views::to_utf_or_error<ToChar>;
  for (auto it = view.begin(), end = view.end(); it != end; ++it) {
    if ((*it).has_value()) {
      result.push_back(**it);
    } else {
      throw std::runtime_error("error at position " +
                               std::to_string(it.base() - input.begin()) + ": " +
                               enum_to_string((*it).error()));
    }
  }
  return result;
}
```

```
  // prints: "error at position 2: truncated_utf8_sequence"
  transcode_or_throw<char8_t, char16_t>(
    u8"hi🙂"sv | std::views::take(5) | std::ranges::to<std::u8string>());
```

§ 6.5 Changing the Suits of Unicode Playing Card Characters

```
enum class suit : std::uint8_t {
  spades = 0xA,
  hearts = 0xB,
  diamonds = 0xC,
  clubs = 0xD
};

// Unicode playing card characters are laid out such that changing the second least
// significant nibble changes the suit, e.g.
// U+1F0A1 PLAYING CARD ACE OF SPADES
// U+1F0B1 PLAYING CARD ACE OF HEARTS
constexpr char32_t change_playing_card_suit(char32_t card, suit s) {
  if (U'\N{PLAYING CARD ACE OF SPADES}' <= card && card <= U'\N{PLAYING CARD KING OF
      CLUBS}') {
    return (card & ~(0xF << 4)) | (static_cast<std::uint8_t>(s) << 4);
  }
  return card;
```

```
}

void change_playing_card_suits() {
  std::u8string_view const spades = u8"♠♡♢♣♤♥♦♧🂡🂱🃁🃑";
  std::u8string const hearts =
    spades |
    to_utf32 |
    std::views::transform(std::bind_back(change_playing_card_suit, suit::hearts)) |
    to_utf8 |
    std::ranges::to<std::u8string>();
  assert(hearts == u8"♥♡♦♢♤♠♧♣🂱🂡🃑🃁");
}
```

§ 6.6 Handling Byte Offsets and Endianness

Say we want to handle a set of bytes in a message starting at offset N with length K that is UTF16BE text:

```
std::u8string parse_message_subset(
    std::span<std::byte> message, std::size_t offset, std::size_t length) {
  return std::span{message.begin() + offset, message.begin() + offset + length}
         | std::views::chunk(2)
         | std::views::transform(
             [](const auto chunk) {
               std::array<std::byte, 2> a{};
               std::ranges::copy(chunk, a.begin());
               return std::bit_cast<std::uint16_t>(a);
             })
         | std::views::from_big_endian
         | std::views::as_char16_t
         | std::views::to_utf8
         | std::ranges::to<std::u8string>();
}
```

Note that this depends on P4030R0 “Endian Views” for `std::views::from_big_endian`.

§ 7 Dependencies

The code unit views depend on [P3117R1] “Extending Conditionally Borrowed”.

§ 8 Implementation Experience

The most recent revision of this paper has a reference implementation called beman.utf_view available on GitHub, which is a fork of Jonathan Wakely’s implementation of P2728R6 as an implementation detail for libstdc++. It is part of the Beman project.

Versions of the interfaces provided by previous revisions of this paper have also been implemented, and re-implemented, several times over the last 5 years or so, as part of a proposed (but not yet accepted!) Boost library, Boost.Text. Boost.Text has hundreds of stars on GitHub.

Both libraries have comprehensive tests.

§ 9 Wording

§ 9.1 Additional Helper Concepts

Add the following to 25.5.2 [range.utility.helpers]:

```
template<class T>
  concept code-unit =
        same_as<remove_cv_t<T>, char8_t> || same_as<remove_cv_t<T>, char16_t> ||
      same_as<remove_cv_t<T>, char32_t>;
```

§ 9.2 Transcoding views

Add the following subclause to 25.7 [range.adaptors]:

§ 24.7.? Transcoding views [range.transcoding]

§ 24.7.?.1 Overview [range.transcoding.overview]

`to_utf_view` produces a view of the UTF code units transcoded from the elements of a *utf-range*. It transcodes from UTF-N to UTF-M, where N and M are each one of 8, 16, or 32. N may equal M. `to_utf_view`'s `ToType` template parameter is based on a mapping between character types and UTF encodings, which is that that `char8_t` corresponds to UTF-8, `char16_t` corresponds to UTF-16, and `char32_t` corresponds to UTF-32. If its `to_utf_view_error_kind` CTP is `expected`, it produces a view of `expected<charN_t, utf_transcoding_error>` where invalid input subsequences result in errors.

The names `views::to_utf<ToType>`, `views::to_utf_or_error<ToType>`, `views::to_utf8`, `views::to_utf8_or_error`, `views::to_utf16`, `views::to_utf16_or_error`, `views::to_utf32`, and `views::to_utf32_or_error` denote range adaptor objects ([range.adaptor.object]).

`views::to_utf<ToType>` is equivalent to `views::to_utf8` if `ToType` is `char8_t`, `views::to_utf16` if `ToType` is `char16_t`, and `views::to_utf32` if `ToType` is `char32_t`, and similarly for `views::to_utf_or_error`.

Let `views::to_utfN` denote any of the aforementioned range adaptor objects, let `Char` be its corresponding character type, and let `Error` be its corresponding `to_utf_view_error_kind`. Let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `decltype((E))` does not model *utf-range*, or if `T` is an array of `char8_t`, `char16_t`, or `char32_t`, `to_utfN(E)` is ill-formed. The expression `to_utfN(E)` is expression-equivalent to:

- If `E` is a specialization of `empty_view` ([range.empty.view]):
  - If `Error` is `to_utf_view_error_kind::replacement`, then `empty_view<Char>{}`.
  - Otherwise, `empty_view<expected<Char, utf_trancoding_error>>{}`.
- Otherwise, if the type of `E` is a (possibly cv-qualified) specialization of `to_utf_view`, then `to_utf_view(E.base(), cw<Error>, to_utf_tag<Char>)`.
- Otherwise, if the type of `E` is *cv* `subrange<to_utf_view::iterator, to_utf_view::iterator, subrange_kind::unsized>` for some specialization of `to_utf_view`, then `to_utf_view(subrange(E.begin().base(), E.end().base()), cw<Error>, to_utf_tag<Char>)`.
- Otherwise, `to_utf_view(E, cw<Error>, to_utf_tag<Char>)`.

§ 24.7.?.2 Enumeration `utf_transcoding_error` [range.transcoding.error.transcoding]

```
enum class utf_transcoding_error {
  truncated_utf8_sequence,
```

```
  unpaired_high_surrogate,
  unpaired_low_surrogate,
  unexpected_utf8_continuation_byte,
  overlong,
  encoded_surrogate,
  out_of_range,
  invalid_utf8_leading_byte
};
```

§ 24.7.?.3 Enumeration `to_utf_view_error_kind` [range.transcoding.error.kind]

```
enum class to_utf_view_error_kind : bool {
  replacement,
  expected
};
```

§ 24.7.?.4 UTF Tags [range.transcoding.tags]

```
template<code-unit ToType>
struct to_utf_tag_t {
  explicit to_utf_tag_t() = default;
};

template<code-unit ToType>
constexpr to_utf_tag_t<ToType> to_utf_tag{};

using to_utf8_tag_t = to_utf_tag_t<char8_t>;

constexpr to_utf8_tag_t to_utf8_tag{};

using to_utf16_tag_t = to_utf_tag_t<char16_t>;

constexpr to_utf16_tag_t to_utf16_tag{};

using to_utf32_tag_t = to_utf_tag_t<char32_t>;

constexpr to_utf32_tag_t to_utf32_tag{};
```

§ 24.7.?.5 Class template `to_utf_view` [range.transcoding.view]

```
template<input_range V, to_utf_view_error_kind E, code-unit ToType>
  requires view<V> && code-unit<range_value_t<V>>
class to_utf_view : public view_interface<to_utf_view<V, E, ToType>> {
private:
  template<bool>
  struct iterator; // exposition only
  template<bool>
  struct sentinel; // exposition only

  V base_ = V(); // exposition only

public:
  constexpr to_utf_view() requires default_initializable<V> = default;
  template <auto E2>
       constexpr  explicit  to_utf_view(V  base,  constant_wrapper<E2,  to_utf_view_er-
         ror_kind>, to_utf_tag_t<ToType>)
       requires (constant_wrapper<E2, to_utf_view_error_kind>::value == E);

  constexpr V base() const& requires copy_constructible<V> { return base_; }
  constexpr V base() && { return std::move(base_); }
```

```
  constexpr iterator<false> begin();
  constexpr iterator<true> begin() const requires range<const V> && ((same_as<range_-
        value_t<V>, char32_t>) || (!forward_range<const V>))
  {
    if constexpr (bidirectional_range<const V>) {
      return iterator<true>(begin(base_), begin(base_), end(base_));
    } else {
      return iterator<true>(begin(base_), end(base_));
    }
  }

  constexpr sentinel<false> end() { return sentinel<false>(end(base_)); }
  constexpr iterator<false> end() requires common_range<V>
  {
    if constexpr (bidirectional_range<V>) {
      return iterator<false>(begin(base_), end(base_), end(base_));
    } else {
      return iterator<false>(end(base_), end(base_));
    }
  }
  constexpr sentinel<true> end() const requires range<const V>
  {
    return sentinel<true>(end(base_));
  }
  constexpr iterator<true> end() const requires common_range<const V>
  {
    if constexpr (bidirectional_range<const V>) {
      return iterator<true>(begin(base_), end(base_), end(base_));
    } else {
      return iterator<true>(end(base_), end(base_));
    }
  }

  constexpr bool empty() const { return empty(base_); }

  constexpr size_t size()
            requires   sized_range<V>   &&   same_as<char32_t,   range_value_t<V>>   &&
        same_as<char32_t, ToType>
  {
    return size(base_);
  }

  constexpr auto reserve_hint() requires approximately_sized_range<V>;
  constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
};

template<class R, auto E2, code-unit ToType>
             to_utf_view(R&&,      constant_wrapper<E2,      to_utf_view_error_kind>,
        to_utf_tag_t<ToType>)
            ->   to_utf_view<views::all_t<R>,   constant_wrapper<E2,   to_utf_view_er-
        ror_kind>::value, ToType>;

template <class V, to_utf_view_error_kind E, class ToType>
     inline   constexpr   bool   enable_borrowed_range<to_utf_view<V,   E,   ToType>>   =
        enable_borrowed_range<V>;
```

```
template <auto E2>
    constexpr   explicit   to_utf_view(V   base,   constant_wrapper<E2,   to_utf_view_er-
        ror_kind>, to_utf_tag_t<ToType>)
    requires (constant_wrapper<E2, to_utf_view_error_kind>::value == E);
```

Effects: Initializes *base_* with `std::move(base)`.

```
constexpr iterator begin();
```

Returns: {*this, std::ranges::begin(base_)}

```
constexpr auto reserve_hint() requires approximately_sized_range<V>;
```

Returns: The result is implementation-defined.

```
constexpr auto reserve_hint() const requires approximately_sized_range<const V>;
```

Returns: The result is implementation-defined.

[*Note:* The implementation of the `empty()` member function provided by the transcoding views is more efficient than the one provided by `view_interface`, since `view_interface`'s implementation will construct `to_utf_view::begin()` and `to_utf_view::end()` and compare them, whereas we can simply use the underlying range's `empty()`, since a transcoding view is empty if and only if its underlying range is empty. — *end note*]

§ 24.7.?.6 Class `to_utf_view::iterator` [range.transcoding.iterator]

```
template<input_range V, to_utf_view_error_kind E, code-unit ToType>
  requires view<V> && code-unit<range_value_t<V>>
template<bool Const>
class to_utf_view<V, E, ToType>::iterator {
private:
  using Base = maybe-const<Const, V>; // exposition only

public:
  using iterator_concept = see below;
  using iterator_category = see below;  // not always present
   using  value_type  =  conditional_t<E  ==  to_utf_view_error_kind::expected,  expect-
        ed<ToType, utf_transcoding_error>, ToType>;
  using reference_type = value_type;
  using difference_type = ptrdiff_t;

private:
  iterator_t<Base> begin_{}; // exposition only, present only if
                             //   bidirectional_range<Base> is true
  iterator_t<Base> current_{}; // exposition only
  sentinel_t<Base> end_; // exposition only

  inplace_vector<value_type, 4 / sizeof(ToType)> buf_{}; // exposition only

  int8_t buf_index_{}; // exposition only
  uint8_t to_increment_{}; // exposition only

  template<input_range V2, to_utf_view_error_kind E2, code-unit ToType2>
    requires view<V2> && code-unit<range_value_t<V2>>
  friend class to_utf_view; // exposition only

public:
  constexpr iterator() requires default_initializable<iterator_t<Base>> = default;

  constexpr iterator(iterator const&) requires copyable<iterator_t<Base>> = default;
   constexpr iterator& operator=(iterator const&) requires copyable<iterator_t<Base>>
        = default;
  constexpr iterator(iterator&&) = default;
  constexpr iterator& operator=(iterator&&) = default;

private:
```

```
    constexpr iterator(iterator_t<Base> begin, iterator_t<Base> current, sentinel_t<Base> end) // exposition only
      requires bidirectional_range<Base>
      : begin_(std::move(begin)), current_(std::move(current)), end_(end) {
      if (base() != end())
        read();
    }

    constexpr iterator(iterator_t<Base> current, sentinel_t<Base> end) // exposition only
      requires (!bidirectional_range<Base>)
      : current_(std::move(current)), end_(end) {
      if (base() != end())
        read();
      else if constexpr (!forward_range<Base>) {
        buf_index_ = -1;
      }
    }

public:
  constexpr const iterator_t<Base>& base() const& noexcept { return current_; }

  constexpr iterator_t<Base> base() && { return std::move(current_); }

  constexpr value_type operator*() const;

  constexpr iterator& operator++() requires (E == to_utf_view_error_kind::expected)
  {
    if (!success()) {
      if constexpr (is_same_v<ToType, char8_t>) {
        advance-one();
        advance-one();
      }
    }
    advance-one();
    return *this;
  }

    constexpr iterator& operator++() requires (E == to_utf_view_error_kind::replacement)
  {
    advance-one();
    return *this;
  }

  constexpr auto operator++(int) {
    if constexpr (is_same_v<iterator_concept, input_iterator_tag>) {
      ++*this;
    } else {
      auto retval = *this;
      ++*this;
      return retval;
    }
  }

  constexpr iterator& operator--() requires bidirectional_range<Base>
  {
    if (!buf_index_)
      read-reverse();
    else
      --buf_index_;
    return *this;
```

```
  }

  constexpr iterator operator--(int) requires bidirectional_range<Base>
  {
    auto retval = *this;
    --*this;
    return retval;
  }

   friend constexpr bool operator==(const iterator& lhs, const iterator& rhs) re-
        quires equality_comparable<iterator_t<Base>>
  {
    return lhs.current_ == rhs.current_ && lhs.buf_index_ == rhs.buf_index_;
  }

private:
  constexpr sentinel_t<Base> end() const { // exposition only
    return end_;
  }

  constexpr expected<void, utf_transcoding_error> success() const noexcept requires(E
        == to_utf_view_error_kind::expected); // exposition only

  constexpr void advance-one() // exposition only
  {
    ++buf_index_;
    if (buf_index_ == buf_.size()) {
      if constexpr (forward_range<Base>) {
        buf_index_ = 0;
        advance(current_, to_increment_);
      }
      if (current_ != end()) {
        read();
      } else if constexpr (!forward_range<Base>) {
        buf_index_ = -1;
      }
    }
  }

  constexpr void read(); // exposition only

  constexpr void read-reverse(); // exposition only
};
```

[*Note:* `to_utf_view::iterator` does its work by adapting an underlying range of code units. We use the term "input subsequence" to refer to a potentially ill-formed code unit subsequence which is to be transcoded into a code point `c`. Each input subsequence is decoded from the UTF encoding corresponding to *`from-type`*. If the underlying range contains ill-formed UTF, the code units are divided into input subsequences according to Substitution of Maximal Subparts, and each ill-formed input subsequence is transcoded into a `U+FFFD`. `c` is then encoded to `ToType`'s corresponding encoding, into an internal code unit buffer `buf_`. — *end note*]

[*Note:* `to_utf_view::iterator` maintains invariants on `base()` which differ depending on whether it's an input iterator. In both cases, if `*this` is at the end of the range being adapted, then `base()` == *`end()`*. But if it's not at the end of the adapted range, and it's an input iterator, then the position of `base()` is always at the end of the input subsequence corresponding to the current code point. On the other hand, for forward and bidirectional iterators, the position of `base()` is always at the beginning of the input subsequence corresponding to the current code point. — *end note*]

`to_utf_view::iterator::iterator_concept` is defined as follows:

— If `V` models `bidirectional_range`, then `iterator_concept` is `bidirectional_iterator_tag`.

— Otherwise, if `V` models `forward_range`, then `iterator_concept` is `forward_iterator_tag`.

— Otherwise, `iterator_concept` is `input_iterator_tag`.

The member *typedef-name* `iterator_category` is defined if and only if `V` models `forward_range`.

In that case, `to_utf_view::iterator::iterator_category` is defined as follows:

— Let `C` denote the type `iterator_traits<iterator_t<V>>::iterator_category`.

— If `C` models `derived_from<bidirectional_iterator_tag>`, then `iterator_category` denotes `bidirectional_iterator_tag`.

— Otherwise, if `C` models `derived_from<forward_iterator_tag>`, then `iterator_category` denotes `forward_iterator_tag`.

— Otherwise, `iterator_category` denotes `C`.

```
constexpr value_type operator*() const;
```

Returns: Either `buf_[buf_index_]`, or, if `E` is `to_utf_view_error_kind::expected` and `!success()`, then `unexpected{success().error()}`

```
constexpr expected<void, utf_transcoding_error> success() const noexcept requires(E
        == to_utf_view_error_kind::expected); // exposition only
```

Returns:

— If *from-type* is `char8_t`:

  — If the current input subsequence is a code unit between 0x80 and 0xBF, returns `unexpected_utf8_continuation_byte`.

  — If the current input subsequence is a code unit between 0xC0 and 0xC2, or between 0xF5 and 0xFF, returns `invalid_utf8_leading_byte`.

  — If the current input subsequence is the code unit 0xE0, and the subsequent input subsequence is a code unit between 0x80 and 0x9F; or if the current input subsequence is the code unit 0xF0, and the subsequent input subsequence is a code unit between 0x80 and 0x8F; then returns `overlong`.

  — If the current input subsequence is the code unit 0xED, and the subsequent input subsequence is a code unit between 0xA0 and 0xBF, then returns `encoded_surrogate`.

  — If the the current input subsequence is the code unit 0xF4, and the subsequent input subsequence is a code unit between 0x90 and 0xBF, then returns `out_of_range`.

  — Otherwise, if the current input subsequence is invalid UTF-8, begins with a code unit between 0xC2 and 0xF4, and there exists some hypothetical sequence of code units which would make the current input subsequence well-formed if concatenated to the end of it, then returns `truncated_utf8_sequence`.

— If *from-type* is `char16_t`:

  — If the current input subsequence is a code unit between 0xD800 and 0xDBFF, returns `unpaired_high_surrogate`.

— If the current input subsequence is a code unit between 0xDC00 and 0xDFFF, returns `unpaired_low_surrogate`.

— If *from-type* is `char32_t`:

— If the current input subsequence is between 0xD800 and 0xDFFF, returns `encoded_surrogate`.

— If the current input subsequence is between 0x110000 and 0xFFFFFFFF, returns `out_of_range`.

Otherwise, returns `expected<void, utf_transcoding_error>()`.

```
constexpr void read(); // exposition only
```

Effects:

Decodes the input subsequence starting at position `base()` into a code point `c`, using the UTF encoding corresponding to *from-type*, and setting `c` to U+FFFD if the input subsequence is ill-formed. It sets `to_increment_` to the number of code units read while decoding `c`. encodes `c` into `buf_` in the UTF encoding corresponding to `ToType`, and sets `buf_index_` to `0`. If `forward_iterator<I>` is `true`, `base()` is set to the position it had before `read` was called.

```
constexpr void read-reverse(); // exposition only
```

Effects:

Decodes the input subsequence ending at position `base()` into a code point `c`, using the UTF encoding corresponding to *from-type*, and setting `c` to U+FFFD if the input subsequence is ill-formed. It sets `to_increment_` to the number of code units read while decoding `c`; encodes `c` into `buf_` in the UTF encoding corresponding to `ToType`; and sets `buf_index_` to `buf_.size() - 1`, or to `0` if this is an `or_error` view and we read an invalid subsequence.

§ 24.7.?.7 Class `to_utf_view::sentinel` [range.transcoding.sentinel]

```
template<input_range V, to_utf_view_error_kind E, code-unit ToType>
  requires view<V> && code-unit<range_value_t<V>>
template<bool Const>
struct to_utf_view<V, E, ToType>::sentinel {
private:
  using Base = maybe-const<Const, V>; // exposition only

  sentinel_t<Base> end_ = sentinel_t<Base>(); // exposition only

public:
  sentinel() = default;
  constexpr explicit sentinel(sentinel_t<Base> end) : end_{end} {}
  constexpr explicit sentinel(sentinel<!Const> i)
    requires Const && convertible_to<sentinel_t<V>, sentinel_t<Base>>
    : end_{i.end_} {}

  constexpr sentinel_t<Base> base() const { return end_; }

  template<bool OtherConst>
  requires sentinel_for<sentinel_t<Base>, iterator_t<maybe-const<OtherConst, V>>>
  friend constexpr bool operator==(const iterator<OtherConst>& x, const sentinel& y)
        {
    if constexpr (forward_range<Base>) {
      return x.current_ == y.end_;
```

```
    } else {
      return x.current_ == y.end_ && x.buf_index_ == -1;
    }
  }
};
```

§ 9.3 Code unit adaptors

Add the following subclause to 25.7 [range.adaptors]:

§ 24.7.? Code unit adaptors [range.codeunitadaptor]

```
template<class T>
struct implicit-cast-to { // exposition only/
  constexpr T operator()(auto x) const noexcept { return x; }
};
```

The names `as_char8_t`, `as_char16_t`, and `as_char32_t` denote range adaptor objects ([range.adaptor.object]). Let `as_charN_t` denote any one of `as_char8_t`, `as_char16_t`, and `as_char32_t`. Let `Char` be the corresponding character type for `as_charN_t`, let `E` be an expression and let `T` be `remove_cvref_t<decltype((E))>`. If `ranges::range_reference_t<T>` does not model `convertible_to<Char>`, or if `T` is an array, `as_charN_t(E)` is ill-formed. The expression `as_charN_t(E)` is expression-equivalent to:

— If `T` is a specialization of `empty_view` ([range.empty.view]), then `empty_view<Char>{}`.

— Otherwise, `ranges::transform_view(std::views::all(E), implicit-cast-to<Char>{})`.

[Example 1:

```
std::vector<int> path_as_ints = {U'C', U':', U'\x00010000'};
std::filesystem::path      path      =      path_as_ints      |      as_char32_t      |
        std::ranges::to<std::u32string>();
const auto& native_path = path.native();
if (native_path != std::wstring{L'C', L':', L'\xD800', L'\xDC00'}) {
  return false;
}
```

— end example]

§ 9.4 Feature test macro

Add the following macro definition to 17.3.2 [version.syn], header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_unicode_transcoding 20XXXXL // also in <ranges>
```

§ 10 Design Discussion and Alternatives

§ 10.1 CPO Rejection of String Literals

String literals are arrays of char types that include a null terminator:

```
static_assert(std::is_same_v<std::remove_reference_t<decltype("foo")>,          const
        char[4]>);
static_assert(std::ranges::equal("foo", std::array{'f', 'o', 'o', '\0'}));
```

Because they are ranges, a naive implementation of the `to_utfN` CPO would result in null terminators in the output:

```
u8"foo" | to_utf32 | std::ranges::to<u32string>()
// results in a std::u32string of length 4 containing U'f', U'o', U'o', U'\0'
```

To avoid this situation, the `to_utfN` CPOs reject all inputs that are arrays of `char`, as do the `as_charN_t` casting CPOs.

§ 10.2 The `_or_error` Views Are Basis Operations for Other Error Handling Behaviors

You can use the `_or_error` view to implement the same behavior that the non-`std::expected`-based views have.

For example, `foo | std::views::to_utf8` has the same output as:

```
foo
  | std::views::to_utf8_or_error
  | std::views::transform(
      [](std::expected<char8_t, std::utf_transcoding_error> c)
        -> std::inplace_vector<char8_t, 3>
      {
        if (c.has_value()) {
          return {c.value()};
        } else {
          // U+FFFD
          return {u8'\xEF', u8'\xBF', u8'\xBD'};
        }
      })
  | std::views::join
```

You can also substitute a different replacement character by changing the result of the `else` clause, or add exception-based error handling by throwing at that point.

§ 10.3 Why We Don't Cache `begin()`

When we invoke `begin()`, constructing the transcoding iterator may read up to four elements from the underlying view if it's transcoding from UTF-8. A previous revision of this paper implemented `begin()` caching, based on the idea that iterating the underlying range could have unbounded complexity.

However, Tim Song pointed to the wording in `[iterator.requirements.general]` stating that "All the categories of iterators require only those functions that are realizable for a given category in constant time (amortized)." This means that we should be making the assumption that the underlying iterator operations used by `begin()` are "amortized constant time" (in a hand-wavey sense). Tim also pointed out that transcoding from UTF is equivalent to `views::adjacent<4>`, which doesn't cache.

Based on this reasoning, the transcoding views don't cache `begin()`.

§ 10.4 Optimizing for Double-Transcoding

In generic code, it's possible to introduce transcoding views that wrap other transcoding views:

```
void foo(std::ranges::view auto v) {
#ifdef _MSC_VER
  windows_function(v | std::views::to_utf16);
```

```
#endif
  // ...
}

int main(int, char const* argv[]) {
  foo(std::null_term(argv[1]) | std::views::as_char8_t | std::views::to_utf32);
}
```

In the above example, naively, `foo` would create a `to_utf16_view` wrapping a `to_utf32_view`. However, the `to_utfN` CPOs detect this situation and elide the `to_utf32_view`, creating the `to_utf16_view` so that it directly wraps the view produced by `as_char8_t`.

There's precedent for this kind of approach in the `views::reverse` CPO, which simply gives back the original underlying view if it detects that it's reversing another `reverse_view`.

§ 11 Changelog

§ 11.1 Changes since R11

— Refactor use of `std::nontype` to use `std::constant_wrapper` instead per [P3948R1].

— Remove back-pointer to parent view in iterator, restoring R7's three-iterator design and borrowedness, based on precedent from `views::adjacent<N>`; and based on the fact that, since R10 implements the double-transcode optimization in the CPO, we no longer need the *`innermost-iter`* machinery that overcomplicated the R7 design.

§ 11.2 Changes since R10

— Fix the wording around rejecting arrays for the code unit adaptors

— Add example for handling byte offsets and endianness

§ 11.3 Changes since R9

— Replace bool OrError CTP with `to_utf_view_error_kind` enum class like ranges::subrange_kind

— Replace `to_utfX_view` classes with a single `to_utf_view` class with annoying constructor tags to make CTAD work, per SG9 feedback from Kona 2025, and remove related obsolete design discussion

— Fix a bug where the value type of `empty_view` was not set properly in the CPO when using an `_or_error` CPO

— Remove section stating that the concept of a CPO template is a "novelty" after it was pointed out in Kona 2025 that it has precedent in `views::adjacent_transform<N>` and elsewhere

— Don't cache `begin()`

— Reject arrays of `charN_t` in the CPOs

— Implement double-transcode optimization in the CPOs

§ 11.4 Changes since R8

— Fix const correctness bug in wording relating to noncopyable input iterators

— Use a std::inplace_vector instead of a std::array for buf_, allowing us to eliminate buf_last_ exposition-only member

— Simplify the implementation of *to-utf-view-impl*'s operators

— Rename *utf-iterator* to *to-utf-view-impl*::*iterator*

— Add a *to-utf-view-impl*::*sentinel* type

— Cache `begin()`

— Clean up wording

— Add `reserve_hint()` member functions

— Fix bug in `operator==` for input iterators

— Add additional design discussion

— Add `size()` member function when transcoding from and to UTF-32

§ 11.5 Changes since R7

— Add playing card example.

— Remove `iterator_interface` from *utf-iterator*.

— Replace code unit views with range adaptor closure objects that are expression-equivalent to P3117 `transform_view`.

— Move `null_sentinel` and `null_term` into P3705.

— Remove `std::uc` namespace and replace it with `std::ranges` and `std::ranges::views`.

— Remove library EB.

— Change iterator to use a back-pointer to its parent view, removing borrowedness and quadratic stack size growth.

— Remove support for `char` and `wchar_t`.

— Rewrite most of the verbiage and add new diagrams.

§ 11.6 Changes since R6

— Fix a bug in `null_sentinel_t` causing it not to satisfy `sentinel_for` by changing its `operator==` to return `bool`.

— Fix a bug in `null_sentinel_t` where it did not support non-copyable input iterators by having operator== take input iterators by reference.

— Rename `as_utfN` to `to_utfN` to emphasize that a conversion is taking place and to contrast with the code unit views, which remain named `as_charN_t`.

— Refactor `utf_view` into an exposition-only *utf-view-impl* class used as an implementation detail of separate `to_utf8_view`, `to_utf16_view`, and `to_utf32_view` classes, addressing broken deduction guides in the previous revision.

— Remove `project_view` and copy most of its implementation into separate `char8_view`, `char16_view`, and `char32_view` classes, addressing broken deduction guides in the previous revision.

— Change `utf_iterator` to an exposition-only member class of *utf-view-impl*.

— Eliminate iterator unpacking mechanism and replace it with an alternative solution to the problem of transcoding ranges wrapping other transcoding ranges. This simplifies the API at the expense of

removing the transcoding iterator's `begin()` and `end()` member functions and losing the ability to implement unpacking for user-defined UTF iterators.

— Remove `std::uc::format`.

— Make all concepts exposition-only.

— Remove `utf_transcoding_error_handler` mechanism.

— Introduce new error handling mechanism based on a new `utf_transcoding_error` enumeration which is returned by an `success()` member function of the transcoding view's iterator.

— Remove ability to pass pointers to range adaptor closure objects, which violated the restriction that only ranges may be passed to range adaptor closure objects.

— Remove `std::format` and `std::ostream` functionality. It doesn't make sense for this mechanism to be the only way we have to format/output `char8_t`; we can revisit this functionality when we have already figured out how to support e.g. `std::u8string`.

— Replace code examples with new ones reflecting API changes.

— Provide a reference implementation.

§ 11.7 Changes since R5

— Simplify the complicated constraint on the comparison operator for `null_sentinel_t`.

— Introduce `ranges::project_view`, and implement `charN_views` in terms of that.

— Convert the `utfN_views` to aliases, rather than individual classes.

§ 11.8 Changes since R4

— Replace `unpacking_owning_view` with `unpacking_view`, and use it to do unpacking, rather than sometimes doing the unpacking in the adaptor.

— Ensure `const` and non-`const` overloads for `begin` and `end` in all views.

— Move `null_sentinel_t` to `std`, remove its `base` member function, and make it useful for more than just pointers, based on SG-9 guidance.

§ 11.9 Changes since R3

— Changed the definition of the `code_unit` concept, and added `as_charN_t` adaptors.

— Removed the utility functions and Unicode-related constants, except `replacement_character`.

— Changed the constraint on `utf_iterator` slightly.

— Change `null_sentinel_t` back to being Unicode-specific.

§ 11.10 Changes since R2

— Add `noexcept` where appropriate.

— Remove non-essential constants and utility functions, and elaborate on the usage of the ones that remain.

— Note differences from similar elements proposed in [P1629R1].

— Extend the examples slightly.

— Correct an error in the description of the view adaptors' semantics, and provide several examples of their use.

§ 11.11 Changes since R1

— Reintroduce the transcoding-from-a-buffer example.

— Generalize `null_sentinel_t` to a non-Unicode-specific facility.

— In utility functions that search for ill-formed encoding, take a range argument instead of a pair of iterator arguments.

— Replace `utf{8,16,32}_view` with a single `utf_view`.

§ 11.12 Changes since R0

— When naming code points in interfaces, use `char32_t`.

— When naming code units in interfaces, use `charN_t`.

— Remove each eager algorithm, leaving in its corresponding view.

— Remove all the output iterators.

— Change template parameters to `utfN_view` to the types of the from-range, instead of the types of the transcoding iterators used to implement the view.

— Remove all make-functions.

— Replace the misbegotten `as_utfN()` functions with the `as_utfN` view adaptors that should have been there all along.

— Add missing `utf_transcoding_error_handler` concept.

— Turn `unpack_iterator_and_sentinel` into a CPO.

— Lower the UTF iterator concepts from bidirectional to input.

§ 12 Relevant Polls/Minutes

§ 12.1 SG9 review of P2728R9 on 2025-11-06 during Kona 2025

— Minutes

POLL: We want to remove the null terminator of char arrays (if present) so that `u8"abc" | to_utf32` (and `to_utf8`, `to_utf16`) do not include the null terminator in the resulting output.

SF	F	N	A	SA
0	2	2	3	1

Attendance: 10 (2 abstentions)

Author Position: A, F

Outcome: Consensus against

POLL: We want to ban char arrays as input to `to_utfX` to prevent accidental inclusion of the null terminator of a string literal.

SF	F	N	A	SA
4	3	1	0	0

Attendance: 10 (2 abstentions)

Author Position: F, SF

Outcome: Strong consensus in favor

ACTION ITEM: Figure out whether we need to cache `begin()`.

POLL: Simplify the `to_utfX_view` classes by just having a single templated view class similar to `scan_view` (with potentially annoying constructor tags to make CTAD work) and focus on usability with the CPOs only.

SF	F	N	A	SA
0	7	1	1	0

Attendance: 9 (0 abstentions)

Author Position: N

Outcome: Consensus in favor

POLL: We want to optimize nested `to_utf_views`.

SF	F	N	A	SA
1	4	4	0	0

Attendance: 9 (0 abstentions)

Author Position: N

Outcome: Consensus in favor

ACTION ITEM: Come up with more examples where nested `to_utf_views` occur in practice to explore whether a designated CPO approach is feasible.

§ 12.2 Unofficial SG9 review of P2728R7 during Wrocław 2024

SG9 members provided unofficial guidance that the `.success()` member function on the *`utf-iterator`* wasn't workable and encouraged providing views with `std::expected` as a value type.

§ 12.3 SG16 review of P2728R6 on 2023-09-13 (Telecon)

— Minutes

No polls were taken during this review.

§ 12.4 SG16 review of P2728R6 on 2023-08-23 (Telecon)

— Minutes

No polls were taken during this review.

§ 12.5 SG9 review of D2728R4 on 2023-06-12 during Varna 2023

— Minutes

POLL: utf_iterator should be a separate type and not nested within utf_view

SF	F	N	A	SA
1	2	1	0	1

Attendance: 8 (3 abstentions)

# of Authors: 1

Author Position: F

Outcome: Weak consensus in favor

SA: Having a separate type complexifies the API

§ 12.6 SG16 review of P2728R0 on 2023-04-12 (Telecon)

— Minutes

POLL: SG16 would like to see a version of P2728 without eager algorithms.

SF	F	N	A	SA
4	2	0	1	0

Attendance: 10 (3 abstentions)

Outcome: Consensus in favor

POLL: UTF transcoding interfaces provided by the C++ standard library should operate on charN_t types, with support for other types provided by adapters, possibly with a special case for char and wchar_t when their associated literal encodings are UTF.

SF	F	N	A	SA
5	1	0	0	1

Attendance: 9 (2 abstentions)

Outcome: Strong consensus in favor

Author's note: More commentary on this poll is provided in the section "Discussion of whether transcoding views should accept ranges of `char` and `wchar_t`". But note here that the authors doubt the viability of "a special case for char and wchar_t when their associated literal encodings are UTF", since making the evaluation of a concept change based on the literal encoding seems like a flaky move; the literal encoding can change TU to TU.

§ 12.7 SG16 review of P2728R0 on 2023-03-22 (Telecon)

— Minutes

No polls were taken during this review.

POLL: `char32_t` should be used as the Unicode code point type within the C++ standard library implementations of Unicode algorithms.

SF	F	N	A	SA
6	0	1	0	0

Attendance: 9 (2 abstentions)

Outcome: Strong consensus in favor

§ 13 Special Thanks

Zach Laine, for writing revisions one through six of the paper and implementing Boost.Text.

Jonathan Wakely, for implementing P2728R6, and design guidance.

Robert Leahy and Gašper Ažman, for design guidance.

The Beman Project, for helping support the reference implementation.

§ 14 References

[CVE-2007-3917] NVD - CVE-2007-3917.
https://nvd.nist.gov/vuln/detail/CVE-2007-3917

[Definitions] The Unicode Standard, Version 17.0, §3.4 Characters and Encoding.
https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G2212

[N2902] JeanHeyd Meneide. Restartable and Non-Restartable Functions for Efficient Character Conversions, revision 6.
https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2902.htm

[Null-terminated multibyte strings] Null-terminated multibyte strings.
https://en.cppreference.com/w/c/string/multibyte.html

[P1629R1] JeanHeyd Meneide. 2020-03-02. Transcoding the world - Standard Text Encoding.
https://wg21.link/p1629r1

[P2871R3] Alisdair Meredith. 2023-12-18. Remove Deprecated Unicode Conversion Facets From C++26.
https://wg21.link/p2871r3

[P2873R2] Alisdair Meredith, Tom Honermann. 2024-07-06. Remove Deprecated locale category facets for Unicode from C++26.
https://wg21.link/p2873r2

[P3117R1] Zach Laine, Barry Revzin, Jonathan Müller. 2024-12-15. Extending Conditionally Borrowed.
https://wg21.link/p3117r1

[P3948R1] Matthias Kretz. 2026-03-24. constant_wrapper is the only tool needed for passing constant expressions.
https://wg21.link/p3948r1

[Substitution] The Unicode Standard, Version 17.0, §3.9.6 Substitution of Maximal Subparts.
https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G66453

[SubstitutionExamples] The Unicode Standard, Version 17.0, §3.9.6 Substitution of Maximal Subparts. https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G67519

[wesnoth] The Battle for Wesnoth, “fixed a crash if the client recieves invalid utf-8.” https://github.com/wesnoth/wesnoth/commit/c5bc4e2a915ddf53b63f292f587526aaa39a96aa